

# Building Electricity Predictor Report

Jack Lockhart  
The University of Alabama

**Abstract**—Accurate forecasting of building energy consumption is vital for effective campus facilities management and attainment of sustainability goals. However, energy demand is characterized by non linear feature interactions that traditional modeling techniques often fail to capture. This project implements an end to end data science pipeline to predict campus energy use by integrating building metadata with real time weather feeds. I engineered specific time and thermal features, including 3 hour rolling temperatures and Cooling/Heating Degree Hours (CDH/HDH), to explicitly map environmental impacts on building infrastructure. I evaluated a baseline Linear Regression model against an XGBoost regressor using Normalized Mean Absolute Error (NMAE) as my primary metric. My findings demonstrate that while the Linear Regression model underperformed with an NMAE of 64.9%, the XGBoost model successfully identified complex feature interactions and non linear thresholds, achieving a significantly lower NMAE of 18.5%. This improvement in predictive accuracy justifies the use of ensemble learning for large scale energy optimization. I deploy the final solution locally as an interactive dashboard, providing energy use predictions for whatever situation it may be required for.

## I. INTRODUCTION

University campuses operate much like small cities and contain a wide variety of residential, academic, and industrial buildings. Managing energy consumption across these facilities is a major challenge for administration because of high operational costs and environmental impacts. When energy demand spikes unexpectedly, it puts significant strain on the power grid and leads to higher utility bills due to peak load pricing. Accurate predictions for building energy usage are therefore a financial and environmental necessity.

The primary people who benefit from better forecasting are campus facilities managers. With better data, they can make informed decisions about when to run heavy machinery or how to adjust cooling settings to avoid expensive energy peaks. The university as a whole also benefits by meeting sustainability goals and reducing its carbon footprint through more efficient resource management.

Standard methods often fail because they assume energy usage changes at a steady rate as it gets hotter or colder. In reality, building energy use is non linear because systems like central chillers usually kick on all at once when a specific temperature threshold is reached. This project addresses this gap by creating a data pipeline that merges building metadata with weather information to model these complex patterns.

The objective of this work is to compare two different models, which are a simple Linear Regression baseline and an XGBoost ensemble model. I want to see which one handles

non linear energy changes more effectively. My research question asks if ensemble learning methods can improve the accuracy of campus energy forecasts compared to traditional linear models when subjected to real world weather fluctuations.

## II. RELATED WORK

Existing research into building energy consumption often focuses on the challenges of comparing diverse facilities and predicting future demand based on environmental factors. This project is directly inspired by several foundational approaches developed for the Building Data Genome Project 2 on Kaggle [1]. There were many articles about the project and about building energy usage in general. Many of these articles also had code demonstrations to give examples and produce useful information from the data set provided.

One common approach in this field is the normalization of electricity usage into Energy Use Intensity (EUI). As demonstrated in an article by Miller [2], this method allows researchers to compare buildings of different sizes by dividing total energy consumption by square footage. My project adopts this normalization strategy as it provides a fair baseline for evaluating the performance of our models across different building types.

Other researchers have focused specifically on the influence of weather on energy consumption. Another article by Miller used visualizations to show that as outdoor temperatures change, there is a corresponding shift in building energy demand [3]. These findings suggest that weather is a primary driver of non linear energy patterns. My work extends this by not just visualizing these effects but by engineering specific thermal features, such as Cooling Degree Hours (CDH), to explicitly capture these impacts within a prediction model.

Furthermore, I explored a project by Liu [4], that used the same dataset to create 24 hours ahead power prediction to assist with demand response planning. While my project adopts the core idea of day long forecasting, I expanded the scope significantly. Instead of only predicting the next 24 hours from the current moment, my tool allows users to select any date to generate a full 24 hour forecast for that day. This makes the model more flexible for planning purposes.

## III. DATA DESCRIPTION

The main dataset for this project is a campus energy dataset from Kaggle that consists of building metadata and historical energy consumption records combined with external weather

data. The dataset has 1,636 buildings with up to 30 pieces of metadata each.

The target variable for our regression models is the electricity consumption measured in kilowatt hours (kWh). Each building is meant to have a measurement every hour for two years, totaling over 28 million data points. However, many of those points are either NaN or invalid as discussed later.

The building dataset includes specific metadata used for training:

- **Square Footage:** The total floor area of the building, acting as a primary indicator of base energy demand.
- **Primary Space Usage:** A categorical variable for the building's purpose, such as Education or Lodging.
- **Sub Primary Space Usage:** A more specific version of space usage to differentiate between building types.
- **Year Built:** The age of the building, which often correlates with HVAC efficiency.
- **Number of Floors:** The vertical floors of the building.
- **Latitude and Longitude:** Used to obtain precise weather information from external APIs.

To account for environmental factors, I merged this building data with hourly weather records from the Open-Meteo weather API. I chose Open-Meteo because it provides both archives and future predictions. By using latitude and longitude to find the closest weather points, I ensured the weather patterns the models see during training match the data format they receive during live predictions in the dashboard.

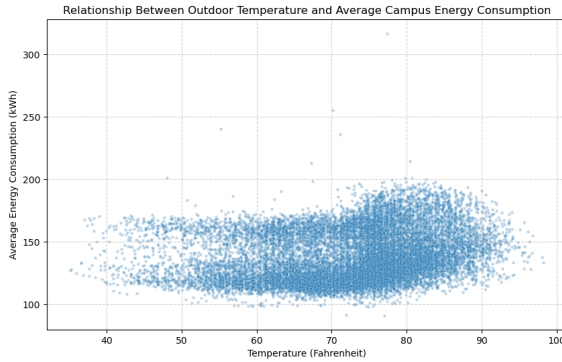


Fig. 1. Relationship between outdoor temperature and average campus energy consumption showing a non linear increase during high heat.

Initial exploratory analysis of the data reveals a clear non linear relationship between temperature and consumption, as shown in Fig. 1. While energy use stays relatively flat during mild temperatures (50°F to 70°F), there is a visible upward curve as temperatures exceed 75°F. This trend represents the activation of large scale cooling systems. The significant scatter in the plot also suggests that temperature alone does not explain all the variance, pointing to the importance of building specific metadata and time of day cycles.

For predicting buildings at the University of Alabama, I needed metadata that was difficult to find online. While names and locations were easy to obtain, things like square footage

and year built were not publicly listed. Since this data is used only for the live demo and not actual training, I utilized AI to generate a list of best guesses for each building's metadata. I manually verified a subset of these buildings to ensure they seemed correct for the final demonstration.

#### IV. METHODOLOGY

My methodology follows a structured data science pipeline designed to transform raw building and weather data into a format suitable for predictive modeling.

##### A. Data Cleaning and Preprocessing

The data cleaning phase was critical for ensuring the integrity of the model training set, as energy data often contains sensor errors and missing records. The following steps were implemented to refine the dataset:

- **Filtering Building Types:** I removed building categories that were not relevant to the building types I was targeting, such as warehouses. This allowed the models to focus on high occupancy types like classrooms and laboratories. This only removed about 100 buildings.
- **Geospatial Filtering:** Buildings without latitude and longitude coordinates were removed. This was necessary because precise location data was required to map buildings to the closest weather stations for accurate weather data. This removed 200 buildings, leaving me with 1285 for now.
- **Zero Value Treatment:** I identified that no functioning campus building has exactly zero energy usage at any given time. These zero values were treated as sensor failures and replaced with NaN placeholders to prevent them from skewing the model. However, this replaced over 900 thousand values with an NaN which is over 5% of the total values.
- **Outlier Removal:** To handle noise, I applied a modified Z score method with a threshold of 3.5. This statistical approach is more robust than a standard Z score as it uses the median absolute deviation, ensuring that genuine energy spikes were preserved while extreme sensor anomalies were removed. However, it was a concern because it created about 670 thousand NaN values, but it was necessary.
- **Gap Interpolation:** For short duration missing data, NaN values, I used linear interpolation to fill gaps of four hours or less. This allowed me to fix many missing data points without introducing large biases. Any longer of a gap would cause issues due to the daily cycle of energy usage. Overall, it helped recover about 600 thousand or about 24% of the NaN values.
- **Data Completeness Threshold:** Finally, any buildings missing more than 10% of their data were removed. That ended up removing 258 buildings, about 20

After completing this cleaning pipeline, I was left with a final dataset of 1027 buildings. This sample size exceeds the minimum requirement of 1,000 samples for the CS 451 project level.

## B. Feature Engineering

The feature engineering stage involved creating new variables to capture the thermal dynamics of building infrastructure and the cyclical nature of energy consumption. My project focuses on three primary engineered features designed to address the non linear relationship between weather and energy demand.

- **Rolling Temperature (3 Hour):** I calculated a rolling average of the outdoor temperature over the previous three hours. This feature was created to account for the thermal mass of large buildings, which do not respond instantaneously to temperature changes but rather absorb and release heat over time. By using a rolling window, the model can better predict energy spikes caused by sustained heat rather than brief fluctuations.
- **Cooling Degree Hours (CDH):** I calculated CDH by taking the outdoor temperature and subtracting a base threshold of 65°F, setting any negative values to zero. This feature explicitly models the energy required to cool a building. It transforms raw temperature into a direct measure of cooling load, which helps the model identify the specific point where HVAC systems must activate.
- **Heating Degree Hours (HDH):** Similar to CDH, HDH was calculated by subtracting the outdoor temperature from the 65°F base. This captures the demand for heating during colder periods. Separating thermal demand into CDH and HDH allows the model to treat heating and cooling as distinct operational modes, which is more representative of real world mechanical systems than using a single temperature column.
- **Cyclical Time Encoding:** I attempted to convert timestamps into circular features using sine and cosine transformations for the hour of the day and month. The intent was to help the model understand that hour 23 is numerically close to hour 0 and December is close to January. However, these features did not provide a significant increase in predictive accuracy. I ultimately relied on raw hour, day, and month categorical features which the ensemble model was able to interpret effectively on its own.

These engineered features align with the Stage 2 project requirements, which mandate at least 3 meaningful features beyond the raw columns. By focusing on thermal accumulation and load thresholds, I provided the models with the necessary information to predict non linear energy loads.

## C. Feature Selection

After the feature engineering phase, I selected a final set of variables that would allow the models to generalize to new data rather than simply memorizing the training set. I prioritized features that represent the physical characteristics of the buildings, environmental conditions, and temporal patterns.

- **Building Traits:** I included **sqft**, **year built**, and **number of floors**. Square footage is the most significant indicator of base energy demand, while the year built provides

context regarding insulation quality and HVAC efficiency. The number of floors helps the model understand the building's volume and vertical scale.

- **Usage Categorization:** Both **primary usage** and **sub primary usage** were selected. These categories are essential because a laboratory or a data center has vastly different energy requirements than a dormitory or a storage facility, even if they have the same square footage.
- **Time Features:** I grouped **hour**, **day of week**, and **month** into our temporal feature set. These capture the daily cycles of human activity, the difference between weekday and weekend operations, and seasonal shifts in energy needs that are not solely caused by temperature.
- **Weather and Thermal Load:** This group includes **temp**, **apparent temp**, and our engineered **temp roll 3**, **CDH**, and **HDH**. While raw temperature gives a snapshot of the environment, the rolling average and degree hour features provide the model with a memory of thermal accumulation and a direct measurement of the cooling or heating load required.
- **Excluded Features:** I explicitly chose to exclude **site id** (the campus name) and **building id**. Including these would have allowed the models to associate specific energy patterns with a unique identifier, essentially memorizing individual buildings. By removing them, I force the models to learn the underlying relationships between physical attributes and energy use, which makes the system much more useful for generalizing to other buildings not seen during training. I also chose to exclude **Circular Time Features** due to the fact that they had almost no correlation on their own and ended up not affecting the outcome of the model at all.

This selection process ensures that my models rely on meaningful drivers, satisfying the requirement to demonstrate domain understanding through thoughtful feature selection.

## D. Models Used and Justification

For this project, I selected two distinct model families to compare how different mathematical approaches handle the complexities of energy data. This satisfies the requirement to train at least two different model families, specifically a linear model and an ensemble method.

- **Linear Regression:** I chose Linear Regression as our baseline model family. It is a highly interpretable model that assumes a direct, constant relationship between input features and the target variable. I used this to establish a performance floor and to see how much accuracy is lost when a model cannot account for the non linear jumps in energy usage that I identified during exploratory data analysis.
- **XGBoost (XGBoosting):** I selected XGBoost as my advanced model because a decision tree based approach is most effective for this problem. Unlike linear models, decision trees excel at capturing step functions and threshold based logic, which is exactly how campus mechanical systems operate. For example, a cooling system usually

kicks on all at once when a specific temperature threshold is reached rather than ramping up slowly. XGBoost is an ensemble method that builds many of these individual trees, where each new tree specifically focuses on correcting the errors made by the previous ones. I chose this specific algorithm because it is much better at picking up on these sudden changes in energy demand caused by weather changes compared to a standard regression line. Additionally, the large scale of my final dataset, which includes over 10 million data points with more than 10 features each, makes a powerful ensemble model like XGBoost a better choice than simpler methods that cannot handle as much complexity.

By comparing these two families, I can demonstrate the clear value that machine learning adds to campus facility management compared to traditional statistical methods.

#### E. Hyperparameter Tuning Strategy

To optimize the performance of the gradient boosting model, I used a multi stage hyperparameter tuning strategy that combined manual experimentation with Bayesian Optimization.

- **Initial Manual Exploration:** I began by manually tuning the parameters using a significantly smaller subset of the data. The goal was to establish a baseline understanding of how variables like tree depth and learning rate affected the model's error. However, I quickly realized that patterns found in a small sample size might not correlate correctly with the behaviors of the full dataset, leading to potential overfitting or poor generalization.
- **Bayesian Optimization with Optuna:** To find a more reliable foundation, I switched to an automated approach using the Optuna framework. I implemented a Bayesian Optimization study that used the Tree structured Parzen Estimator sampler to efficiently search multiple combinations. Due to the massive size of the complete dataset, which would have made my 100 trials take way too long, I ran this optimization on a controlled subset. I searched for optimal values for the **number of leaves**, **regularization (alpha and lambda)**, and **subsampling ratios**. This provided a good baseline for each parameter that was statistically better than my initial manual guesses.
- **Final Calibration:** In the final stage, I used the results from the Bayesian study as a starting point and performed a final manual tuning using the full model and the complete dataset. This allowed me to ensure that parameters like the **number of estimators** and **learning rate** were properly scaled for the full volume of 1,027 buildings. This hybrid approach allowed me to achieve a high degree of accuracy without the insane computational time required to run automated optimization on the entire dataset.

#### F. Evaluation Metrics and Validation Strategy

To ensure the model would perform reliably on data it had never seen before, I implemented multiple validation frame-

works that accounted for both the temporal and categorical nature of the dataset.

- **Evaluation Metrics:** I primarily utilized **Root Mean Squared Error (RMSE)** and **Mean Absolute Error (MAE)** to assess performance. RMSE was particularly important because it squares the residuals, thereby penalizing large outliers more heavily. I used RMSE as the main evaluator because in energy forecasting, a large prediction error can be more costly for grid management than several small ones, making RMSE an ideal metric. I also monitored MAE to understand the average error magnitude in actual kWh units.
- **Time Based Validation:** Since energy consumption is a time series problem, I used a chronological split for my primary validation. This ensured that the model was trained on past data and tested on later data, preventing data leakage where the model might accidentally peek into the future to make a prediction. For the split, the data has 24 months of data so I used 18 months for training and 6 months for validation, being about a 25% validation split.
- **Group Based Validation:** Beyond just time, I implemented a GroupKFold strategy based on building IDs. This was essential to test the model's ability to generalize to entirely new buildings. By ensuring that all data from a specific building was either in the training set or the test set, but never both, I could verify that the model was learning the physical relationship between building features and energy use, rather than just memorizing a specific building's unique fingerprint. For group based split I set it to use 20% of the data as validation.
- **Final Test Split:** For my final model, I used a time split because I believe that having more buildings in the data set will help the model generalize more than training with a group split, even though the group split validates with new, unseen buildings.

## V. RESULTS

The performance of the models was evaluated using a 20% hold out test set. This set was not used during the training or hyperparameter tuning phases, providing an unbiased assessment of how the models handle unseen data across different buildings and time periods.

#### A. Performance Comparison

Table I summarizes the performance of the three primary models: the Linear Regression baseline, the XGBoost forecasting model (Time based split), and the XGBoost generalization model (Group based split).

TABLE I  
MODEL PERFORMANCE COMPARISON

| Model            | RMSE        | CVRMSE       | MAE         | NMAE         |
|------------------|-------------|--------------|-------------|--------------|
| Lin. Reg. (Base) | 217.4       | 147.0%       | 96.0        | 64.9%        |
| XGB (Time)       | <b>65.8</b> | <b>44.5%</b> | <b>27.4</b> | <b>18.5%</b> |
| XGB (Group)      | 176.2       | 113.4%       | 87.1        | 56.0%        |

The XGBoost models both performed better than the linear regression showing that the ensemble was able to capture the non linear data better than the linear regression. However, the XGBoost group split was much worse than the time split because predicting completely new buildings it has never seen before is tougher than predicting previously seen buildings in the future.

### B. Visual Analysis of Model Fit

The parity plots provide a visual diagnostic of the error distributions. As shown in Fig. 2, the Linear Regression model shows weird patterns that look like failure, characterized by horizontal grouping patterns and physically impossible negative energy predictions.

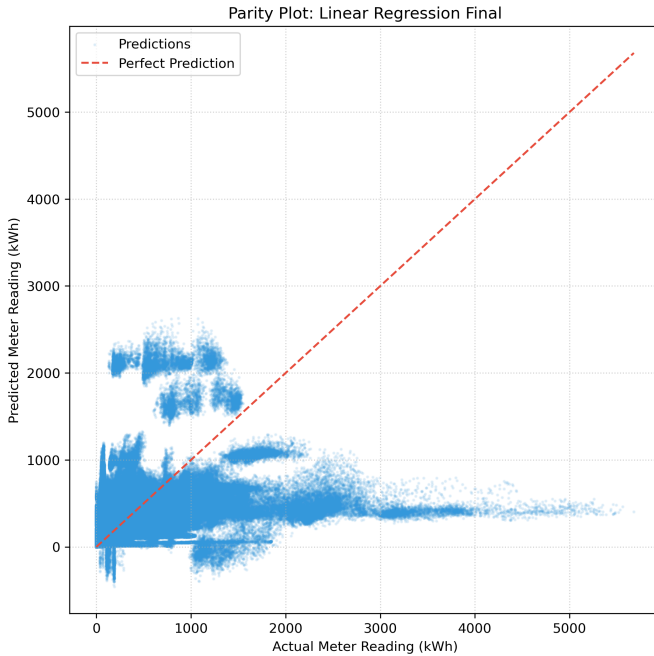


Fig. 2. Parity Plot for Linear Regression Baseline showing significant prediction variance.

In contrast, the XGBoost final model (Fig. 3) shows a much tighter clustering. While average variance increases slightly during large loads, the model maintains a strong correlation across the full range of campus building types.

### C. Feature Importance Analysis

The relative contribution of each feature to the XGBoost models is presented in Table II. The table only includes the most important features. While other features did have an effect on the accuracy of the model, their importances are low. The table shows that my engineered features CDH and 3 hour temperature rolling window are more important than the raw temperature, apparent temperature, and my engineered HDH. I believe that to be because electricity is used more during cooling while other sources such as gas are used more in heating.

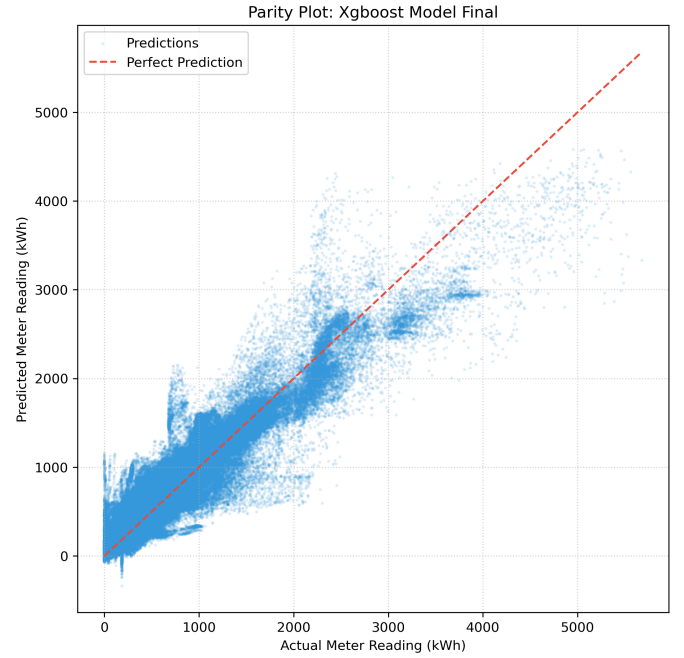


Fig. 3. Parity Plot for XGBoost Final Model demonstrating accurate tracking of energy consumption trends.

TABLE II  
TOP FEATURE IMPORTANCES (XGBOOST)

| Feature             | Importance (Time) | Importance (Group) |
|---------------------|-------------------|--------------------|
| sqft                | 0.2561            | 0.1895             |
| sub_type            | 0.2378            | 0.3093             |
| number_of_floors    | 0.1346            | 0.0737             |
| primary_space_usage | 0.1030            | 0.1870             |
| year_built          | 0.0834            | 0.0621             |
| hour                | 0.0465            | 0.0539             |
| CDH                 | 0.0340            | 0.0341             |
| temp_roll_3h        | 0.0288            | 0.0377             |

## VI. DISCUSSION

The experimental results validate the hypothesis that ensemble learning methods, specifically XGBoost, are superior to traditional linear models for campus energy forecasting. However, the varying performance across different validation strategies and the specific behavior of the models reveal critical insights into the nature of building energy data.

### A. Model Performance

The biggest takeaway is the massive gap between the Linear Regression NMAE of 64.9% and the XGBoost NMAE of 18.5%. The linear model failed because it tries to draw a straight line through data that does not move in a straight line. If you look at the horizontal grouping in the parity plot, it is clear the linear model just guessed the same average number for a bunch of different scenarios. On the other hand, XGBoost uses a tree structure that can handle sudden jumps. In a real building, the AC does not just slowly turn on, it usually kicks in at full power once it hits a certain temperature. XGBoost is great at catching those sudden changes.

There is also a huge difference between predicting the future of a building the model has seen before versus a building that is completely new. The error jumped from 65.8 to 176.2 when switching to the group split. This means every building has its own personality or specific equipment that the general metadata like square footage does not fully explain.

### *B. Features*

The feature importance table confirms that engineering the 3 hour rolling temperature and CDH was worth the effort. These features beat out raw temperature because they give the model a sense of how buildings hold onto heat over time. CDH was a lot more useful than HDH, which makes sense because this dataset is about electricity. Most campus buildings use electricity for cooling but might use gas or steam for heating, which would not show up on these meters.

### *C. Limitations and Biases*

A major limitation here is that the model has no idea what the school calendar looks like. It assumes every Monday is the same, but a Monday during spring break will have way less people than a Monday during finals week. This wouldn't be an issue if every college followed the same exact schedule for breaks and school year activities because the model could learn that, but that is just not possible. One way to fix this would be with occupancy data for each building but that would be a whole separate issue to gather that data.

There is also geographic bias. The data set seemed to have campuses from multiple different climates and even continents, but it didn't have campuses from every possible type of area. The weather features are used to help minimize this but it's not a perfect fix. For example, if the data had no campuses from areas as hot as Alabama, then the model may not predict Alabama buildings well, even with the weather features.

Another concern is building metadata. Buildings get renovated all the time. If a building got a new addition, the square footage would go up but the metadata sqft may not change. Also, the dataset contains 2 years worth of data. In that time, if a building got a more efficient cooling system, the electricity usage could change drastically and the model would have no idea why.

### *D. Validity Risks*

Using the modified Z score to delete outliers was necessary to clean the data, but it is also a risk. By deleting over 600,000 points, I might have accidentally erased real examples of when the energy grid was under extreme stress. This could make the model too conservative, meaning it might fail to predict a massive energy spike during a record breaking heatwave because it learned to treat those spikes as errors.

Also, using interpolation to fill in gaps of four hours or less helped save some data, but it assumes the energy use just moved smoothly from one point to the next. In reality, a lot can happen in four hours, so the training data might be slightly smoothed out in a way that does not perfectly reflect real life.

## VII. CONCLUSION AND FUTURE WORK

This project shows that using machine learning, specifically XGBoost, is a major upgrade over old school linear models for campus energy forecasting. By engineering features like Cooling Degree Hours and rolling temperature averages, I was able to give the model a better understanding of how buildings actually react to heat. The final XGBoost model ended up being much more reliable, dropping the error rate significantly and proving that it can handle the non linear jumps in energy use that happen on a university campus. The results also highlight that predicting the future of a known building is much easier than predicting for a totally new one. While the model is great at forecasting, it still has a ways to go before it can perfectly generalize to any building without historical data.

### *A. Directions for Improvement*

For future work, the most obvious step is using data from the buildings I want to predict. The time split model worked much better than the group split model because the time split just had to predict the future of buildings it already learned rather than having to guess how a new building works. Another improvement could be adding occupancy data. Knowing when students are actually in the classrooms or when the dorms are empty for break would likely fix the biggest over prediction errors. Adding a feature for the official university holiday calendar would also be an easy way to make the model smarter without needing extra sensors.

## VIII. CONTRIBUTIONS

### *A. Individual Contributions*

Since this was a solo project, I was responsible for the entire development lifecycle. I handled the initial data acquisition and weather API integration, developed the cleaning pipeline, and performed the feature engineering for thermal and cyclical variables. I also conducted the exploratory data analysis, managed the model training and Bayesian optimization using Optuna, and built the final FastAPI deployment and interactive dashboard. I used AI to help me speed up the process when needed but I manually verified everything.

### *B. Personal Reflection*

This project gave me experience with the complexities of real world data, especially with all the inconsistencies like sensor failures. I learned how to use basic tools like linear regression but also how to use more advanced tools like XGBoost when linear regression couldn't grasp the data complexities. The most challenging part for me was the data cleaning. The data started with almost a million missing values so I had to balance adding more NaN values while trying to remove as much bad data as possible. If I were to do this project differently, I would attempt to get more data about the buildings I am trying to predict with my model, because without extra data like other heating sources, it is hard to make accurate predictions.

## REFERENCES

- [1] C. Miller, "Building Data Genome Project 2," Kaggle, 2020. [Online]. Available: <https://www.kaggle.com/datasets/claytonmiller/buildingdatagenomeproject2>
- [2] C. Miller, "Comparing Energy from Different Buildings Example," Kaggle, 2020. [Online]. Available: <https://www.kaggle.com/code/claytonmiller/comparing-energy-from-different-buildings-example>
- [3] C. Miller, "Weather Influence on Energy Consumption Example," Kaggle, 2020. [Online]. Available: <https://www.kaggle.com/code/claytonmiller/weather-influence-on-energy-consumption-example>
- [4] D. Liu, "24h Ahead Power Prediction and Demand Response," Kaggle, 2020. [Online]. Available: <https://www.kaggle.com/code/dophineliu/24h-ahead-power-prediction-and-demand-response>

## APPENDIX

### AI Log Entry 1

Date: 2026-3-5 Tool: Gemini

Used by: Jack Phase: Planning

PURPOSE: I needed ideas on good projects to build.

PROMPT: "I have a data science project. I have attached a pdf of the requirements. I am in college and want ideas relating to something with college so I can somewhat personalize it. Give me 5 ideas that are not mentioned in the instructions that I could apply to my university."

AI OUTPUT: 1. Dining Hall Waste Traffic Predictor 2. Campus Safety Lighting Optimization 3. Dormitory Energy Efficiency Auditor 4. Course Enrollment "Seat-Open" Probability 5. Campus "Walkability" Transit Optimizer

YOUR ACTION: I liked idea 3 but wanted something more general. I used it as inspiration for my idea of modeling all building energy usage.

FINAL RESULT: I ended up having the idea of making a model to predict the energy usage of any building on UA campus.

### AI Log Entry 2

Date: 2026-3-5 Tool: Perplexity

Used by: Jack Phase: Planning

PURPOSE: I needed a free weather api with both historical data and forecasting and needed help finding a good one.

PROMPT: "Find me 3 free weather APIs that have both historical data per hour and some forecast."

AI OUTPUT (Summary): 1. Open-Meteo 2. Weatherbit 3. Visual Crossing

YOUR ACTION: I went with Visual Crossing because it had a query builder that I think will make it the easiest.

FINAL RESULT: I chose to use Visual Crossing for my weather API.

### AI Log Entry 3

Date: 2026-3-15 Tool: Gemini

Used by: Jack Phase: Planning

PURPOSE: I didn't do anything specific with this, but it helped me understand the dataset better.

PROMPT: "Explain this dataset. Specifically explain what each row and column means.". I then pasted the first line of the csv which was the column names and the first row of data which was over 1500 data points.

AI OUTPUT (Summary): It is a multivariate time-series dataset. The first column is the timestamp. Every other column is a building with the naming convention [Group]\_[Category]\_[Individual]. The group is the site of buildings. The category is the type of building. The individual is the specific identifier for each building. Each row is a timestamp of an hour with a value for each building at that time.

YOUR ACTION: I didn't do anything specific with this, but it helped me understand the dataset better.

FINAL RESULT: I understood the dataset better, so I was able to work with it and explain it to AI better.

### AI Log Entry 4

Date: 2026-3-15 Tool: Gemini

Used by: Jack Phase: Visualization

PURPOSE: I didn't understand the dataset I had obtained and needed it explained.

PROMPT: "Explain this dataset. Specifically explain what each row and column means.". I then pasted the first line of the csv which was the column names and the first row of data which was over 1500 data points.

AI OUTPUT (Summary): It is a multivariate time-series dataset. The first column is the timestamp. Every other column is a building with the naming convention [Group]\_[Category]\_[Individual]. The group is the site of buildings. The category is the type of building. The individual is the specific identifier for each building.

YOUR ACTION: I didn't do anything specific with this but it helped me understand the dataset better.

FINAL RESULT: I understood the dataset better so I was able to work with it and explain it to AI better.

### AI Log Entry 5

Date: 2026-3-15 Tool: Gemini

Used by: Jack Phase: Visualization

PURPOSE: I needed to see the basic data shape. For example the number of columns and rows, unique values, missing values, counts for each value, etc.

PROMPT: Give me a jupyter notebook code cell that will load my data from "../data/electricity\_clean.csv" and the metadata from "../data/metadata.csv". Then, log important information such as the number of columns and rows, important unique values, and total missing values from the data set.

AI OUTPUT (Summary): This is only some of the code it gave. 1. Get the number of buildings (columns) num\_buildings = len(df\_elec.columns) 2. Get the number of timestamps (rows) num\_timestamps = len(df\_elec.index) 3. Get the date range for context start\_date = df\_elec.index.min().strftime('end\_date = df\_elec.index.max().strftime('usage\_counts = df\_names['Usage'].value\_counts())

YOUR ACTION: I used the output it gave and made a plot to see the distribution of buildings by usage type to ensure that I had enough of the types of buildings I need.

FINAL RESULT: The code worked well. I was able to gather a lot of information from it and was able to choose my next visualization step easily

---

AI Log Entry 6

---

Date: 2026-3-21 Tool: Gemini Used by: Jack Phase: Visualization

PURPOSE: I wanted graphs for energy usage over the year and by day of the week and wanted to see if AI could help me do it faster or at least give me a starting point for it.

PROMPT: After the previous cell you just gave me, give me a new code cell to show a graph of the energy usage by building type over the total time frame. Then, give me a code cell that will give the energy usage for each day of the week. This should include every Monday, Tuesday, etc, to find the average energy usage on that day. Finally, give me a code cell to find the average energy usage per hour of day. It should be the average energy usage of that hour from the whole data set. For each of these, only plot education, lodging, public, and assembly buildings for now.

AI OUTPUT (Summary): It gave a ton of code but it all worked pretty well. It made code for 5 graphs total.

YOUR ACTION: I had to do a tiny bit of debugging and then started switching around the building types to see if there were any sort of relationships between buildings to see if any of them acted similarly.

FINAL RESULT: I was able to gather a lot of information. For example, I hadn't thought of it yet, but energy usage drops a lot on weekends.

---

AI Log Entry 7

---

Date: 2026-3-21 Tool: Gemini Used by: Jack Phase: Cleaning

PURPOSE: I did not know how to remove buildings that were missing a lat and lon so I had AI do it for me.

PROMPT: I created a new jupyter notebook and in the first cell I load my data with "My loading code". Remember the format that the electricity and metadata is in. Write a cell that will remove all buildings that don't have a lat or lon.

AI OUTPUT (Summary): Gave some code to remove buildings with lat/lon. Here are 2 key lines. "mask\_has\_coords = df\_metadata['lat'].notnull()df\_metadata['lng'].notnull()valid\_index = df\_metadata.loc[mask\_has\_coords,'building\_id'].unique()"

YOUR ACTION: I used the code and ran some tests with it. I had to tweak the code a little because it didn't remember the format of the metadata correctly.

FINAL RESULT: The code worked well and removed all the correct buildings when I tested it and was pretty fast.

---

AI Log Entry 8

---

Date: 2026-3-21 Tool: Gemini Used by: Jack Phase: Cleaning

PURPOSE: I was able to do most of the other cleaning manually because it was mostly removing bad data, but I did not know how to interpolate.

PROMPT: Now, using the input df "df\_no\_outliers", interpolate missing values from the electricity data of up to a given

gap size. Use a linear interpolation. Give me the full code of the cell.

AI OUTPUT (Summary): Gave some code to interpolate data which was actually really simple. This was the important line. "df\_interpolated = df\_no\_outliers.interpolate(method='linear', limit=GAP\_SIZE, limit\_direction='both')"

YOUR ACTION: I tested the code and it worked well right away. I had to tweak the gap size to what I think it should be.

FINAL RESULT: The code worked well and interpolated the values correctly.

---

AI Log Entry 9

---

Date: 2026-4-10 Tool: Gemini Used by: Jack Phase: Feature Engineering

PURPOSE: My data was in an unusable wide format and I needed a format I could combine other data with.

PROMPT: I have a new jupyter notebook. I loaded my cleaned data using "My loading code". Write a full code cell to convert the data from a wide format into one that is usable with training a model as well as combining with other data such as weather.

AI OUTPUT (Summary): It gave me code that melted it from wide to long. Here is the key part: "df\_long = df\_start.reset\_index().melt( id\_vars='timestamp', var\_name='building\_id', value\_name='meter\_reading' )"

YOUR ACTION: I tested the code and had to change some of the column names because it kept forgetting some and not knowing when I changed some others during cleaning.

FINAL RESULT: The code worked well and converted my data into a more usable format.

---

AI Log Entry 10

---

Date: 2026-4-10 Tool: Gemini Used by: Jack Phase: Feature Engineering

PURPOSE: I needed to combine my weather data with the electricity data and engineer a few features.

PROMPT: Now, using that new formatted data, combine it with the weather data from "df\_weather". It has temperature and apparent\_temperature. Then engineer cooling degree hours and heating degrees hours from 65 degree Fahrenheit. Finally, change the feature rolling temp 3 which is the rolling temp over the last 3 hours.

AI OUTPUT (Summary): It gave me code that engineered the features then combined it with the electricity dataset. Here is the key part: "df\_weather['CDH'] = (df\_weather['temperature\_f'] - 65).clip(lower=0) df\_weather['HDH'] = (65 - df\_weather['temperature\_f']).clip(lower=0) df\_weather['temp\_rol\_3h'] = df\_weather.groupby('location\_id')['temperature\_f'].transform(lambda x: x.rolling(window=3).mean() )"

YOUR ACTION: I had to fix some of the column names of stuff as usual but then it worked.

FINAL RESULT: The code worked and engineered my 3 features and added the weather data to my electricity data.

---

AI Log Entry 11

Date: 2026-4-10 Tool: Gemini Used by: Jack Phase: Feature Engineering

PURPOSE: I wanted to make circular time features.

PROMPT: Now, convert the time features, hour, day of week, and month into circular features to allow for the model to understand that December is next to January.

AI OUTPUT (Summary): It gave me code that created the circular time features. Here is the important part: `df_time['hour_sin'] = np.sin(2 * np.pi * df_time['hour'] / 24)` `df_time['hour_cos'] = np.cos(2 * np.pi * df_time['hour'] / 24)`

YOUR ACTION: It actually got it all right this time. I checked the output to test that it got it all correct but it seems to work.

FINAL RESULT: The code worked and engineered my circular time features for me.

---

---

AI Log Entry 12

Date: 2026-4-12 Tool: Gemini Used by: Jack Phase: Training

PURPOSE: I need to start the training phase and did not know where to start code wise.

PROMPT: Im starting the training part in a new notebook. I loaded the data and did basic preprocessing with “;Loading code;”. First, I want you to make the target the EUI of the building. Next, allow me to add a list of features to exclude during training so I can test each one. Finally, create a split of data for training and validation. Have one that splits the data based on the time and one that splits random buildings off. Try to use about an 80/20 split.

AI OUTPUT (Summary): It gave me code that didn’t work until the second try. The important piece is “`train_mask = df['timestamp'] ; time_split val_mask = df['timestamp'] ;`” `time_split`

`X_train, y_train = df.loc[train_mask, features], df.loc[train_mask, target]` `X_val, y_val = df.loc[val_mask, features], df.loc[val_mask, target]”`

YOUR ACTION: I had to reprompt the AI because the code did not work the first time, and it was not just a basic column issue that I know exactly how to fix. I then tested the code to make sure there was a pretty good split.

FINAL RESULT: The code worked after the second prompt and gave me the correct splits and target and features.

---

---

AI Log Entry 13

Date: 2026-4-12 Tool: Gemini Used by: Jack Phase: Training

PURPOSE: I needed to train a linear regression model.

PROMPT: Now, using those splits, train a linear regression model using sklearn. Give me the full code cell and print out the final coefficients and validation results.

AI OUTPUT (Summary): It gave me code that worked correctly. The important part was: “`model_lr.fit(X_train_lr, y_train)”`

YOUR ACTION: I did not really have to do anything except interpret the results. I was kind of confused at first because it was so bad but at least I had a baseline to tweak for later.

FINAL RESULT: The code worked and I had a baseline that worked with training so now I could test different feature combinations.

---

---

AI Log Entry 14

Date: 2026-4-12 Tool: Gemini Used by: Jack Phase: Training

PURPOSE: I needed to train an XGBoost model.

PROMPT: Now, using those splits again, train an XGBoost model. Explain the validation method and whatever parameters you decide to include. Print out the results, especially the importance of each feature and the accuracy of the model. Give me the full code cell.

AI OUTPUT (Summary): It gave me code that worked correctly because it was honestly somewhat simple using XGBoosts training. Here is the model and evaluation: “`model = xgb.XGBRegressor(**params)` `preds_eui = model.predict(X_val)` `preds_raw = preds_eui * X_val['sqft']”`

YOUR ACTION: I had to install XGBoost to my jupyter notebook which was pretty easy. I think had to interpret the results since the code actually worked first try.

FINAL RESULT: The code worked and I had another baseline model that I could start tweaking to make a real working model.

---